Lunar Lander

CS 7642: Reinforcement Learning and Decision Making
Corey Elowsky
celowsky3@gatech.edu
Github Hash: 8d4e22dd3a6fd8a8279a10cb400bea6738402732

Abstract: *This paper aims to solve the "Lunar Lander" problem in OpenAI gym using a Deep Q Network. We will investigate the effect of three different hyperparameters on its training and performance.*

1. Introduction: It is well accepted that Reinforcement Learning algorithms can be separated into two categories - model free and model based. Model free methods differ from model based in that model free algorithms assume no prior knowledge of the reward landscape and must learn only through the consequences of the agent's actions. For the Lunar Lander problem, since the agent has no knowledge of potential rewards we are forced to take a model free approach. There are several model free reinforcement algorithms such as TD(lambda), Q Learning, and Policy Gradient, but for this problem I have chosen to use a Deep Q Network (DQN) due to its ability to represent high dimensional observations via a deep neural network to estimate the Q function. In this report I will investigate the effects of three hyperparameters on the training and performance of DQN to solve the Lunar Lander problem - hidden layer size, epsilon decay factor, and replay buffer size.

2. Lunar Lander: Lunar Lander is a rocket trajectory optimization problem. In this environment the agent takes the form of a moon landing aircraft whose goal is to safely land on a landing pad on the surface of the moon. Figure 1 below shows a screenshot of the environment.

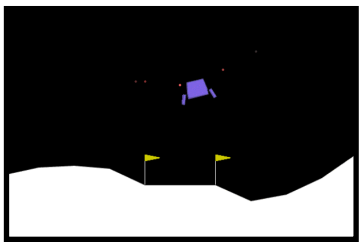

Figure 1: Screenshot of Lunar Lander Environment

The lander begins each episode at the top center of the screen and is given a random initial force applied to its center of mass. The landing pad always remains at the same location, but the surface of the moon changes shape each episode. There are 4 actions that the agent can take, and a state space consisting of an 8-dimensional vector which are described in Figure 2 below.

Actions
Do Nothing
Fire Left Engine
Fire Main Engine
Fire Right engine

States
x position
y position
x velocity
y velocity
angle
angular velocity
left leg contact
right leg contact

Figure 2: Actions and States

All states are represented by continuous values, except for the two leg contact states which are boolean. The reward structure is based on various factors. Firing the main engine and the side engine generates rewards of -0.3 and 0.03 respectively. Crashing on the surface leads to a reward of -100 and landing safely earns a +100 reward. Each leg contacting the ground also receives a reward of +10 points. Finally, an episode terminates if the lander crashes, the lander goes outside the screen, or the agent lands on the surface.

3. Algorithm/Model: In order to solve the Lunar Lander problem I chose to implement a Deep Q Network (DQN) inspired by the pseudocode below in Figure 3.

Initialize replay memory D to capacity N
Initialize action-value function Q with random weights θ
Initialize target action-value function $\hat{Q}$ with weights $\theta^{-} = \theta$
For episode $= 1, M$ **do**
  Initialize sequence $s_1 = \{x_1\}$ and preprocessed sequence $\phi_1 = \phi(s_1)$
  For $t = 1$,T **do**
    With probability ε select a random action a_t
    otherwise select $a_t = \mathrm{argmax}_a Q(\phi(s_t), a; \theta)$
    Execute action a_t in emulator and observe reward r_t and image x_{t+1}
    Set $s_{t+1} = s_t, a_t, x_{t+1}$ and preprocess $\phi_{t+1} = \phi(s_{t+1})$
    Store transition $(\phi_t, a_t, r_t, \phi_{t+1})$ in D
    Sample random minibatch of transitions $(\phi_j, a_j, r_j, \phi_{j+1})$ from D
    Set $y_j = \begin{cases} r_j & \text{if episode terminates at step j+1} \\ r_j + \gamma \max_{a'} \hat{Q}\left(\phi_{j+1}, a'; \theta^{-}\right) & \text{otherwise} \end{cases}$
    Perform a gradient descent step on $\left(y_j - Q\left(\phi_j, a_j; \theta\right)\right)^2$ with respect to the network parameters θ
    Every C steps reset $\hat{Q} = Q$
  End For
End For

Figure 3: DQN Pseudocode

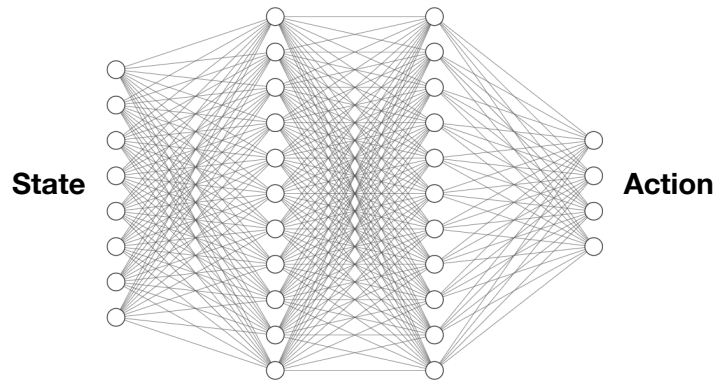

Figure 4: Neural Network Architecture

The main difference between DQN and more traditional reinforcement learning algorithms is that DQN uses a neural network to decide which action to take as opposed to a conventional Q-table. Since this problem does not have a finite state space (e.g. the state variables are continuous) using a Q-table would require potentially error introducing simplifications such as discretizing the state space, and I decided that it would be more useful to use a neural network in its place instead. Since a neural network is a non-linear function approximator it can essentially be used to learn any function, given enough hidden layers. Although many recent DQN papers use convolutional networks to directly extract information from the game screen to predict actions, I decided that this may be unnecessary, and decided to use a neural network with only fully connected layers that attempts to map a state to an action. I chose to use an architecture with two hidden layers, that takes an eight dimensional vector as input (the current state) and a four dimensional vector as output, where each output node represents the Q value of each action. The output node with the largest value would then correspond to the action that should be taken given the current state. Each hidden layer is constrained to be the same size, but the actual size is a hyperparameter that I will tune in the following sections. Below in Figure 4 I show the network architecture.

The simplest way to train a DQN would be to update the weights of the neural network after every time-step with the state, next state, and action values from that time-step in what is known as standard online Q-learning. However this has several disadvantages since there are strong correlations between consecutive samples, making training very inefficient. As a better approach I use what is known as experience replay. In this approach a replay buffer is created which stores a finite amount of state, action, next, state, reward experiences. The network is then only trained after a predefined set of time-steps, and is trained with a mini-batch of samples which are sampled uniformly at random from the replay buffer. Since we are randomly sampling from a buffer with a fixed size, this allows the network to be trained multiple times with the most recent experiences, which if the network is learning, will be more reward rich, leading to more efficient learning. Another important part of my implementation included actually two neural networks. Once which is used to make predictions, and one which is only used to the target action values. Every C time-steps all parameters in the target neural network are replaced with the parameters in the acton value prediction neural network. Since the targets will be computed using an older set of parameters this allows for a more stable learning process and helps prevent the likelihood of divergence and oscillation in performance.

4: Hyperparameter Optimization:

(i) Hidden Layer Size

The first hyperparameter that I chose to explore is the hidden layer size. The neural network layer has two hidden layers which I arbitrarily constrained to be the same size for simplification purposes. In fully connected layers the hidden layer size determines how many parameters the network has to learn and represent, and therefore has a profound effect on the training and performance. A network with a small hidden layers size will be able to train and predict quickly due to the small amount of parameters, but may not be complex enough to model the complex non-linear relationship between the states and the actions. However, a network with too many parameters will take a very long time to train and predict, and also may overfit to the training data and be unable to generalize to new data. Below in Figure 5 I show the results of training for varying hidden layers sizes.

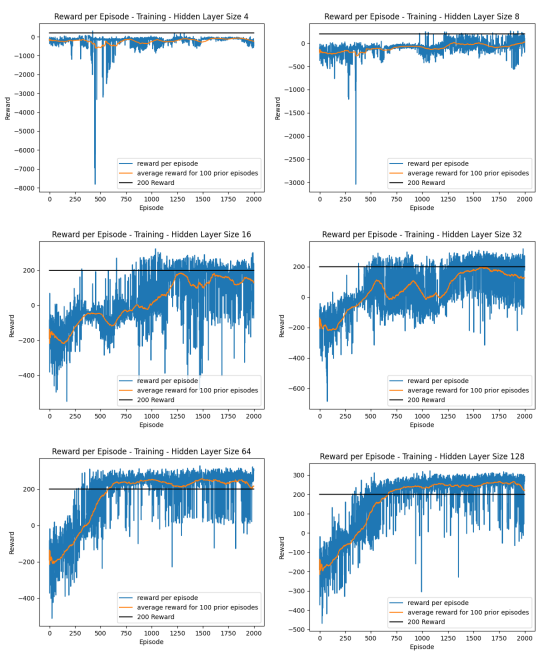

Figure 5: Hidden Layer Size Optimization

We can see from the training plots above that when the hidden layer size is too small (4,8,16), the DQN is unable to learn a policy that meets the reward threshold of 200 after 2000 episodes. When the hidden layer size is increased to both 64 and 128 the reward threshold is met and becomes fairly stable after about 500 episodes. In the further experiments I will choose to go with the smaller hidden layer size of 64 since it uses significantly less parameters than the size of 128, and therefore takes less time to train and is less likely to overfit.

(ii) Epsilon Decay

Another important hyperparameter is the epsilon decay factor. I chose to use an epsilon greedy approach for choosing an action in order to allow the agent to both explore its environment by choosing random actions, but also leverage what has been learned already by using the Q neural network to choose an action. Since more exploration is needed earlier in the training I start epsilon at 1 so that it will always explore randomly, and then after each episode I decay epsilon by multiplying the current epsilon by some epsilon decay factor that is less than 1. I also set the minimum that epsilon can be to 0.1 so that the agent will always be allowed to explore even late in training so that it will be less likely to overfit. Below in Figure 6 I show the epsilon decay schedule for varying values of decay factors.

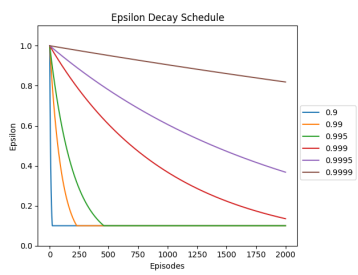

Figure 6: Epsilon Decay Schedules

Choosing this epsilon decay factor is a tradeoff. If the factor is too large, epsilon will decay very slowly, leading to too much random exploration and the agent could take a very large amount of episodes to learn. However, if the epsilon decay is too small, epsilon will decay too quickly, and the agent will not have explored the environment enough to observe all the different reward

rich states. Below in figure 7 I show the results of training for the different values of epsilon decay, using the optimal hidden layer size of 64 found previously.

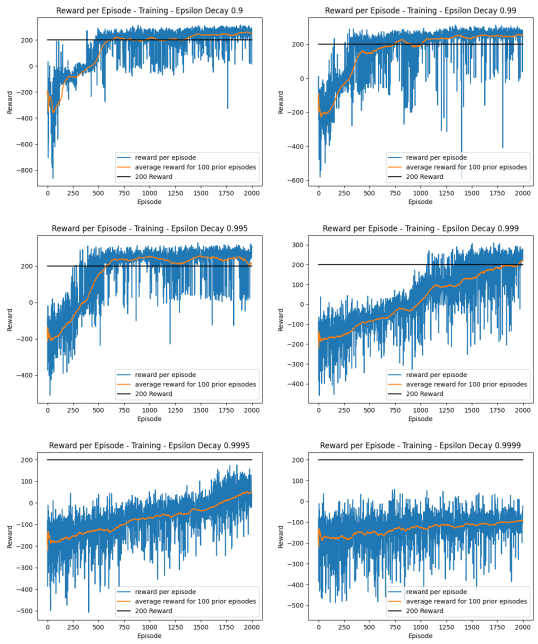

Figure 7: Epsilon Decay Optimization

As it can be seen in the plots above when the epsilon decay factor is too large (e.g. >= .999) the agent is unable to meet the reward threshold after 2000 episodes. Even though the average reward is trending upward and will most likely reach the threshold eventually, the training is very slow due to the agent randomly exploring too much and not taking advantage of the experience learned through the neural network. It appears that the optimal epsilon decay factor is .995 and this will be the value used in further experiments.

(iii) Replay Buffer Size

As discussed earlier, the experience replay buffer is a fixed size buffer used to store the agent's most recent state, action, reward, next state experiences. To train the model a minibatch of experiences are sampled uniformly at random from this buffer. Since this buffer is a fixed size, once the buffer reaches maximum capacity, the oldest experience is removed from the buffer and replaced with the most recent experience. If the buffer is too small then the buffer will potentially not contain reward rich experiences that the network could benefit learning from. However if the buffer is too large, the buffer will contain many very old experiences that may not provide much learning value to the network. Below in figure 8 I show the result of training for varying replay buffer sizes using the optimal hidden layer size of 64 and optimal epsilon decay factor of.995 that were found previously.

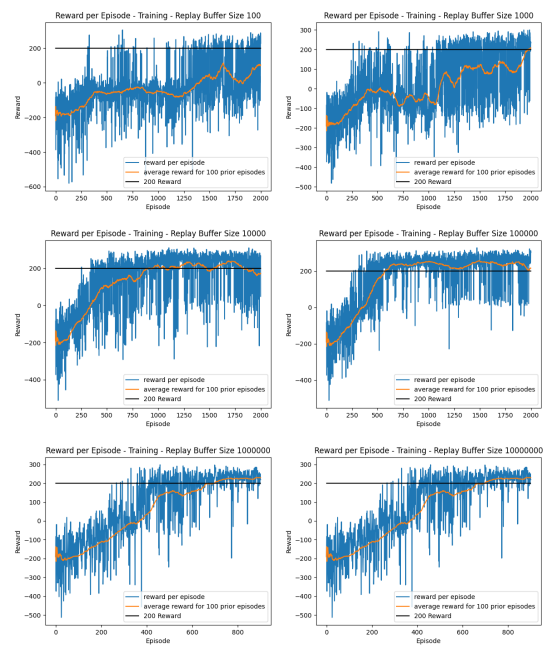

Figure 8: Replay Buffer Size Optimization

As it can be seen in the pilots below, when the buffer size is too small, the training takes many more episodes to converge due to the presence of too many correlated experiences in the buffer, and when the buffer size is too large the training is incredibly slow and inefficient due to the buffer containing too many old experiences. It is clear that the optimal replay buffer size is 10000. With this size the training converges around the 500th episode.

5: Results:

Based on the results of the hyperparameter optimization the final network was trained with the following hyperparameters - hidden

layer size (64), epsilon decay factor (.995), replay buffer size (100000). Below in figure 9 I show the result of training with these hyper-parameters.

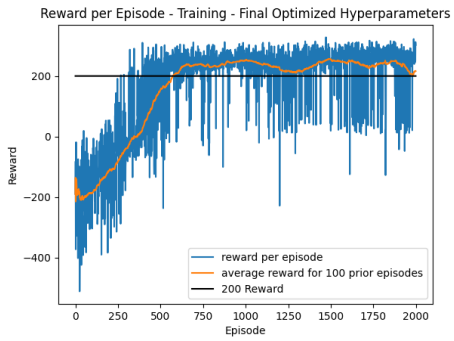

Figure 9: Training with Optimized Hyperparameters

With the optimized hyperparameters we can see that the training is both efficient and accurate. The agent is able to achieve the average reward threshold of 200 after about 500 episodes, and the performance remains relatively stable as it continues to train. However in order to better evaluate the efficacy of the model I will now test the model on 100 consecutive episode where all actions are determines solely by the Q neural network that was trained. Below in Figure 10 we can see the results.

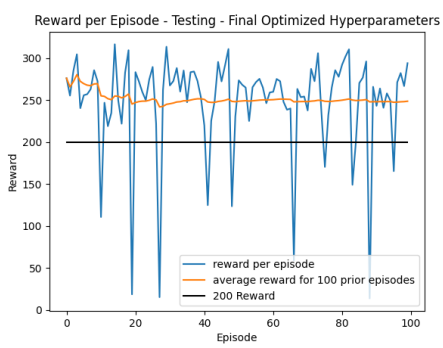

Figure 10: Testing with Final Qnet

In the plot above it can be seen that over 100 episodes 90 of those episodes achieved a reward of at least 200. Although there are 10 episodes that do not meet this threshold, this episodes still achieve a positive reward.

6: Discussion:

After optimizing three optimizers I was able to train a Qnet model that was able to achieve a 90% success rate in reaching the 200 reward threshold. There were many other hyperparameters that could have been trained, but due to computational and time constraints I was not able to optimize these and instead chose values that I thought were reasonable. If I had more time I would certainly perform a much larger gird search in order to optimize all hyperparameters. One of the major pitfalls I experienced was that I first attempted to tune hyperparameters without using experience replay. Initially I thought that even without experience replay I would be able to optimize a network to converge but the training was too unstable and I wasted a large amount of time doing this. Another method I would like try if I had more time would be to implement a more intelligent sampling of the replay buffer instead of using uniform random sampling. Since the buffer is sampled randomly it is possible that many of the experiences samples for the minibatch are not reward rich and therefore don't provide much learning value. I would like to try sampling such that experiences with more extreme rewards would be more likely to be samples in order to speed up training.

7: Conclusion:

Overall this project was a success. I was able to successfully train a DQN to solve the Lunar Lander problem, and in the process was able to better understand how sensitive Deep Q Networks are to hyperparameters.

8 References:

1) Richard S Sutton and Andrew G Barto. Reinforcement learning: An introduction. 2nd Ed. MIT press, 2020. url: http://incompleteideas.net/book/the-book-2nd.html.
2) Playing Atari with Deep Reinforcement Learning, Mnih et al.
3) Human-level control through Deep Reinforcement Learning, Mnih et al.